



Università Politecnica delle Marche

Facoltà di Ingegneria

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea Magistrale in Ingegneria Informatica e
dell'Automazione

Analisi e Predizione di Incidenti di Cybersecurity

Classificazione e Sviluppo di un Modello Predittivo

Docenti

Prof. Ursino Domenico
Dott. Buratti Christopher

Componenti del gruppo

Dott. Tempera Fabio
Dott. Marcianesi Luca
Dott. Vianello Gabriele

ANNO ACCADEMICO 2024-2025

Indice

1	Introduzione	4
1.1	Obiettivi	4
2	Explorative Data Analysis	6
2.1	Panoramica del Dataset	6
2.2	Struttura e Gerarchia dei Dati	6
2.3	Analisi Descrittiva	9
2.3.1	Distribuzione della Variabile Target (IncidentGrade) .	9
2.3.2	Analisi Temporale degli Incidenti	10
2.3.3	Distribuzione delle Gravità degli Incidenti	12
2.3.4	Titole della segnalazione generate	12
2.3.5	Frequenza dei Titoli Maggiori	13
2.3.6	Analisi componenti hardware	14
2.3.7	Analisi geografica	15
2.3.8	Distribuzione Geografica	15
2.3.9	Tecniche MITRE ATT&CK più Frequenti	15
2.3.10	Correlazione tra le Variabili Numeriche	16
2.4	Strumenti Utilizzati	17
3	Extract - Transform - Load	19
3.1	Data Cleaning	19
3.1.1	Eliminazione delle Colonne	19
3.2	Strategia di Splitting	20
3.3	Feature Engineering	21
3.3.1	Smoothed Risk Score	21
3.3.2	Frequency Encoding e Geo-Localizzazione	21
3.3.3	Gestione delle Tecniche MITRE ATT&CK	21
3.4	Aggregazione: Da Evidence a Incident	22
3.5	Bilanciamento e Selezione delle Feature	22
3.5.1	Analisi della Multicollinearità	22
3.5.2	Gestione dello Sbilanciamento (Undersampling) . .	23
3.5.3	Scaling delle Feature Numeriche	24

3.6	Risultati della Fase di Preprocessing	24
4	Classificazione	25
4.1	Metodologia di Valutazione	25
4.2	Modelli di Classificazione	25
4.2.1	Tuning degli Iperparametri	26
4.2.2	Decision Tree	26
4.2.3	Random Forest	27
4.2.4	XGBoost	28
4.2.5	Multi-layer Perceptron	29
4.3	Valutazione Comparativa dei Modelli	32
4.4	Analisi Dettagliata delle Performance	33
4.5	Analisi di Interpretabilità: Feature Importance	34
4.5.1	Predominanza di AlertRisk_max	35
4.5.2	Rilevanza delle Tecniche MITRE ATT&CK	35
4.5.3	Indicatori Comportamentali e di Contesto	36
4.5.4	Conclusioni sul Modello	36
4.6	Valutazione Finale del Modello	36
5	Conclusioni	38
5.1	Riepilogo e Interpretazione dei Risultati	38
5.1.1	La Sfida dei "True Positive"	38
5.1.2	Dinamicità del Dominio e Concept Drift	39
5.2	Contributi Principali del Progetto	39
5.3	Sviluppi Futuri e Prospettive di Ricerca	40

Elenco delle figure

2.1	Distribuzione delle tre classi di IncidentGrade.	9
2.2	Distribuzione di IncidentGrade in formato binario (TruePositive vs Non-TruePositive).	10
2.3	Distribuzione degli incidenti per giorno della settimana, suddivisi per IncidentGrade.	11
2.4	Distribuzione degli incidenti per ora del giorno, suddivisi per IncidentGrade.	12
2.5	Distribuzione della gravità degli incidenti.	12
2.6	Conteggio alertTitle	13
2.7	Matrice di correlazione tra le variabili numeriche del dataset.	17
3.1	Correlazioni delle principali feature con il target.	23
3.2	Correlazioni delle principali feature con il target.	24
4.1	Metriche di classificazione dell'albero decisionale.	27
4.2	Metriche di classificazione della Random Forest.	28
4.3	Metriche di classificazione di XGBoost.	29
4.4	Metriche di classificazione del Multi-layer Perceptron.	30
4.5	Curve di apprendimento del modello.	31
4.6	Prestazioni sul test set	31
4.7	Confronto delle performance dei quattro modelli su cinque metriche chiave: Accuracy, Precision, Recall, F1-Score e ROC AUC. Si osserva come i modelli basati su alberi (Random Forest, XGBoost) tendano a sovraperformare il modello MLP.	32
4.8	Confronto dei punteggi ROC AUC per i modelli binari. XGBoost emerge come il modello con la più alta capacità discriminante (AUC = 0.9304), seguito da vicino da Random Forest e Decision Tree.	33
4.9	Matrici di confusione per i modelli Random Forest, Decision Tree, XGBoost e MLP.	34
4.10	10 feature più importanti secondo il modello XGBoost.	35

1 Introduzione

In questo lavoro sono state adottate diverse soluzioni tecniche, per esplorare, analizzare, modellare e classificare i dati contenuti nel Microsoft Security Incident Prediction, Dataset specifico per gli incidenti di sicurezza informatica.

1.1 Obiettivi

Gli obiettivi principali di questo progetto sono la progettazione, lo sviluppo e la valutazione di tecniche di machine learning per la classificazione e previsione di incidenti di cybersecurity. E' stata posta una maggiore attenzione sulla capacità dei modelli nel distinguere i falsi attacchi o eventi non pericolosi da rischi o attacchi reali. L'analisi si basa sul dataset GUIDED (Microsoft), la più grande raccolta pubblicamente disponibile di incidenti di cybersecurity del mondo reale, che offre un'opportunità unica per addestrare e testare modelli su scenari realistici.

Nello specifico, il progetto si articola nei seguenti obiettivi:

- **Esplorazione:** la fase di *Explorative Data Analysis* sarà fondamentale per avere piena comprensione dei dati e delle relazioni tra essi; in questo modo si selezioneranno le caratteristiche più importanti ai fini dell'analisi.
- **Classificazione:** creazione di modelli predittivi in grado di stimare con elevata accuratezza i gradi di valutazione degli incidenti — vero positivo (TP), benigno positivo e falso positivo (Non-TP) — sulla base delle risposte storiche degli analisti e delle caratteristiche telemetriche degli avvisi. L'obiettivo è automatizzare il primo livello di analisi, consentendo agli analisti di concentrare le proprie risorse sulle minacce più critiche e riducendo il tempo medio di risposta.
- **Valutazione:** stabilire un criterio di valutazione standardizzato e riproducibile per i sistemi di risposta guidata utilizzando dati reali. Questo

permetterà di valutare e confrontare oggettivamente le capacità dei modelli sviluppati.

- **Analisi interpretativa:** Esplorare, attraverso tecniche di interpretabilità dei modelli (es. feature importance), i fattori e le caratteristiche che contribuiscono maggiormente alla classificazione di un incidente. Questo obiettivo mira a fornire insight azionabili per i team di sicurezza, rispondendo a domande strategiche come: "Quali tipi di avvisi o tecniche MITRE ATT&CK sono più spesso associati a minacce reali?" o "Quali indicatori telemetrici sono più predittivi di un falso positivo?".

Attraverso l'impiego di queste tecniche, il progetto mira a fornire contributi significativi per una migliore comprensione delle minacce alla sicurezza e per la definizione di interventi di risposta personalizzati ed efficaci. L'obiettivo finale è ridurre il carico di lavoro manuale degli analisti SOC, minimizzare l'errore umano e, in ultima analisi, migliorare la postura di sicurezza complessiva delle organizzazioni.

2 Explorative Data Analysis

2.1 Panoramica del Dataset

Nel contesto della sicurezza informatica, la disponibilità di dati realistici e su larga scala è fondamentale per lo sviluppo e la validazione di nuove tecnologie difensive. A tal fine, Microsoft ha rilasciato **GUIDE (Guided Response Utterance and Incident Data at Enterprise-scale)**, la più grande raccolta pubblicamente disponibile di incidenti di cybersecurity del mondo reale.

GUIDE contiene oltre 13 milioni di prove per 33 tipi di entità, coprendo 1,6 milioni di avvisi e 1 milione di incidenti annotati con etichette di triage dai clienti in un periodo di due settimane. Di questi incidenti, 26.000 contengono etichette aggiuntive relative alle azioni di rimedio intraprese. La telemetria proviene da oltre 6.100 organizzazioni, con 9.100 `DetectorId` unici, personalizzati e integrati in numerosi prodotti di sicurezza, che coprono 441 tecniche uniche del framework MITRE ATT&CK.

L'elevata disponibilità dei dati presenti in GUIDE consente di addestrare modelli con tecniche del Machine e Deep Learning per implementare un efficiente sistema di triage degli avvisi.

2.2 Struttura e Gerarchia dei Dati

Il dataset è organizzato secondo una gerarchia a tre livelli che rispecchia il processo di aggregazione delle informazioni in un ambiente SOC: (1) prove (evidence), (2) avvisi (alerts) e (3) incidenti (incidents).

- **Prove:** Rappresentano il livello più granulare di informazione. Una prova è un singolo artefatto o evento atomico, come un indirizzo IP, un hash di file, un'e-mail o i dettagli di un processo. Ogni prova contiene metadati specifici che la contestualizzano.
- **Avvisi:** Al livello intermedio, gli avvisi consolidano una o più prove correlate per segnalare un potenziale evento di sicurezza. Ad esempio,

un avviso di "Esecuzione di malware sospetto" potrebbe aggregare prove relative a un hash di file, al processo che lo ha eseguito e alle connessioni di rete generate.

- **Incidenti:** Al livello più alto, gli incidenti comprendono uno o più avvisi, rappresentando una narrazione coesa di una potenziale violazione della sicurezza. È a questo livello che l'analista applica l'etichetta di triage finale — *Vero Positivo (TP)*, *Benigno Positivo (BP)* o *Falso Positivo (FP)* — che costituisce la variabile target del nostro studio.

La Tabella 2.1 riassume le colonne principali utilizzate in questo progetto, estratte e aggregate dai diversi livelli del dataset per creare un set di dati unificato a livello di incidente.

Colonna	Formato	Descrizione
Id	Stringa/Intero	Identificativo univoco del record o dell'evento.
OrgId	Stringa	Identificativo univoco dell'organizzazione.
IncidentId	Intero	Identificativo univoco dell'incidente.
AlertId	Stringa	Identificativo univoco dell'avviso.
Timestamp	Datetime	Data e ora dell'evento o della creazione dell'avviso.
DetectorId	Stringa	ID del rilevatore che ha generato l'avviso.
AlertTitle	Stringa	Titolo descrittivo dell'avviso.
Category	Categoriale	Categoria dell'avviso (es. Credential Access, Malware).
MitreTechniques	Lista di stringhe	Tecniche MITRE ATT&CK associate all'incidente.
IncidentGrade	Categoriale	Grado o classificazione dell'incidente.
ActionGrouped	Categoriale	Azione aggregata o raggruppata intrapresa (es. Blocked).
ActionGranular	Stringa	Azione specifica e dettagliata intrapresa.
EntityType	Categoriale	Tipo di entità coinvolta (es. File, Process, IpAddress).
EvidenceRole	Categoriale	Ruolo dell'entità come prova (es. Source, Destination).
DeviceId	Stringa	Identificativo univoco del dispositivo.
Sha256	Stringa (Hash)	Hash SHA256 di un file o di un'altra entità.
IpAddress	Stringa	Indirizzo IP coinvolto nell'evento.
Url	Stringa	URL coinvolto nell'evento.
AccountSid	Stringa	Security Identifier (SID) dell'account utente.

Colonna	Formato	Descrizione
AccountUpn	Stringa	User Principal Name (UPN) dell'account (es. utente@dominio.com).
AccountObjectId	Stringa	Object ID dell'account in Azure Active Directory.
AccountName	Stringa	Nome dell'account utente.
DeviceName	Stringa	Nome del dispositivo (hostname).
NetworkMessageId	Stringa	Identificativo univoco di un messaggio di rete (es. email).
EmailClusterId	Stringa/Intero	ID di un gruppo di email simili o correlate.
RegistryKey	Stringa	Percorso completo di una chiave di registro di Windows.
RegistryValueName	Stringa	Nome del valore all'interno di una chiave di registro.
RegistryValueData	Stringa	Dati contenuti in un valore di registro.
ApplicationId	Stringa	Identificativo univoco di un'applicazione.
ApplicationName	Stringa	Nome dell'applicazione.
OAuthApplicationId	Stringa	ID di un'applicazione OAuth coinvolta.
ThreatFamily	Stringa	Famiglia di minacce o malware associata (es. Zloader).
FileName	Stringa	Nome del file coinvolto.
FolderPath	Stringa	Percorso della cartella contenente il file.
ResourceIdName	Stringa	Nome di una risorsa (es. in un provider cloud).
ResourceType	Stringa	Tipo di risorsa.
Roles	Lista di stringhe	Ruoli associati a un'entità (es. utente, risorsa).
OSFamily	Categoriale	Famiglia del sistema operativo (es. Windows, Linux).
OSVersion	Stringa	Versione specifica del sistema operativo.
AntispamDirection	Categoriale	Direzione del flusso email (es. Inbound, Outbound).
SuspicionLevel	Categoriale	Livello di sospetto associato all'evento.
LastVerdict	Stringa	L'ultimo verdetto o classificazione dell'evento.
CountryCode	Stringa (ISO)	Codice ISO del paese associato (es. a un IP).
State	Stringa	Stato o regione associata.
City	Stringa	Città associata.

2.3 Analisi Descrittiva

L'analisi descrittiva è stata condotta per esplorare e sintetizzare le principali caratteristiche del dataset, fornendo una comprensione preliminare delle variabili, delle loro distribuzioni e delle relazioni reciproche. Questo passaggio è fondamentale per identificare pattern, anomalie e per guidare le successive fasi di pre-processing e modellazione.

2.3.1 Distribuzione della Variabile Target (IncidentGrade)

La variabile target, `IncidentGrade`, definisce la natura di un incidente e può assumere tre valori principali: `BenignPositive`, `TruePositive` e `FalsePositive`. La sua distribuzione è un aspetto cruciale, poiché uno sbilanciamento tra le classi può influenzare le performance e la generalizzazione dei modelli di classificazione. Come illustrato nella Figura 2.1, il dataset mostra un certo grado di sbilanciamento. La classe `BenignPositive` è la più rappresentata con il 43.4% del totale, seguita da `TruePositive` con il 35.1% e infine da `FalsePositive` con il 21.5%. Sebbene non estremo, questo squilibrio suggerisce che un modello potrebbe tendere a favorire la classe maggioritaria.

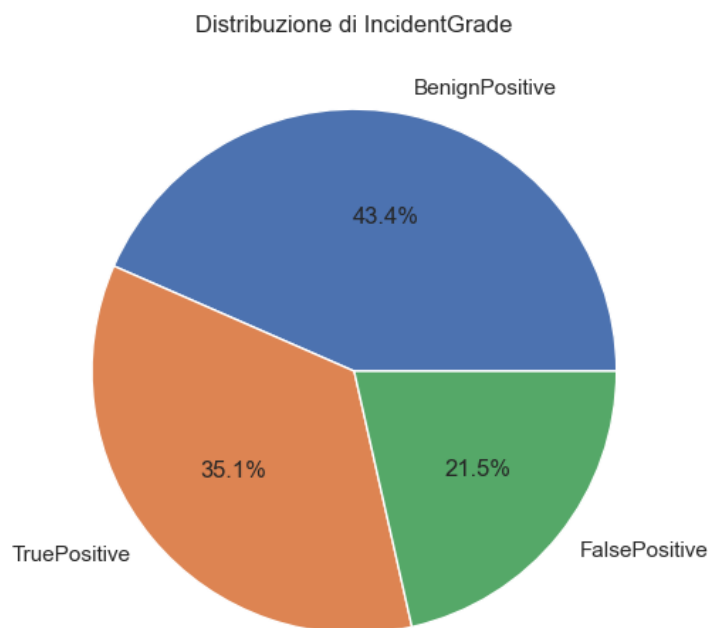


Figura 2.1: Distribuzione delle tre classi di `IncidentGrade`.

Per molti scenari applicativi, l'obiettivo primario è distinguere i veri positivi (`TruePositive`) da tutti gli altri eventi non rilevanti (`Non-TruePositive`,

che aggregano `BenignPositive` e `FalsePositive`). Riformulando il problema in un'ottica binaria, otteniamo la distribuzione mostrata in Figura 2.2. In questa configurazione, i `TruePositive` costituiscono il 40% del dataset, mentre i `Non-TruePositive` rappresentano il restante 60%. Questo sbilanciamento è meno pronunciato e offre una base più stabile per un compito di classificazione binaria.

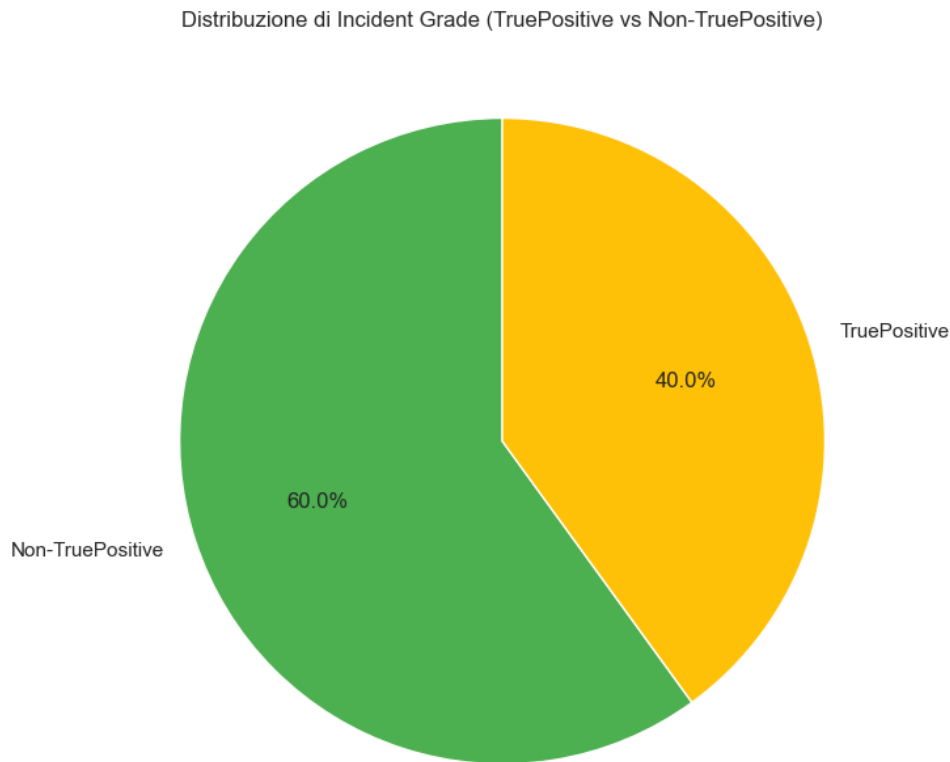


Figura 2.2: Distribuzione di IncidentGrade in formato binario (`TruePositive` vs `Non-TruePositive`).

2.3.2 Analisi Temporale degli Incidenti

L'analisi della distribuzione temporale degli incidenti può rivelare pattern legati alle attività operative. La Figura 2.3 mostra il numero totale di eventi registrati per ogni giorno della settimana, con una scomposizione basata sulla classe di `IncidentGrade`. Emerge chiaramente una concentrazione degli incidenti durante i giorni lavorativi, da lunedì a venerdì, con un picco massimo registrato il martedì. Al contrario, durante il fine settimana (sabato e domenica) si osserva una drastica riduzione del volume di eventi. Questa tendenza è prevedibile e riflette i cicli di attività aziendale. È interessante notare come la proporzione tra le diverse classi di incidenti rimanga relativamente costante durante la settimana lavorativa.

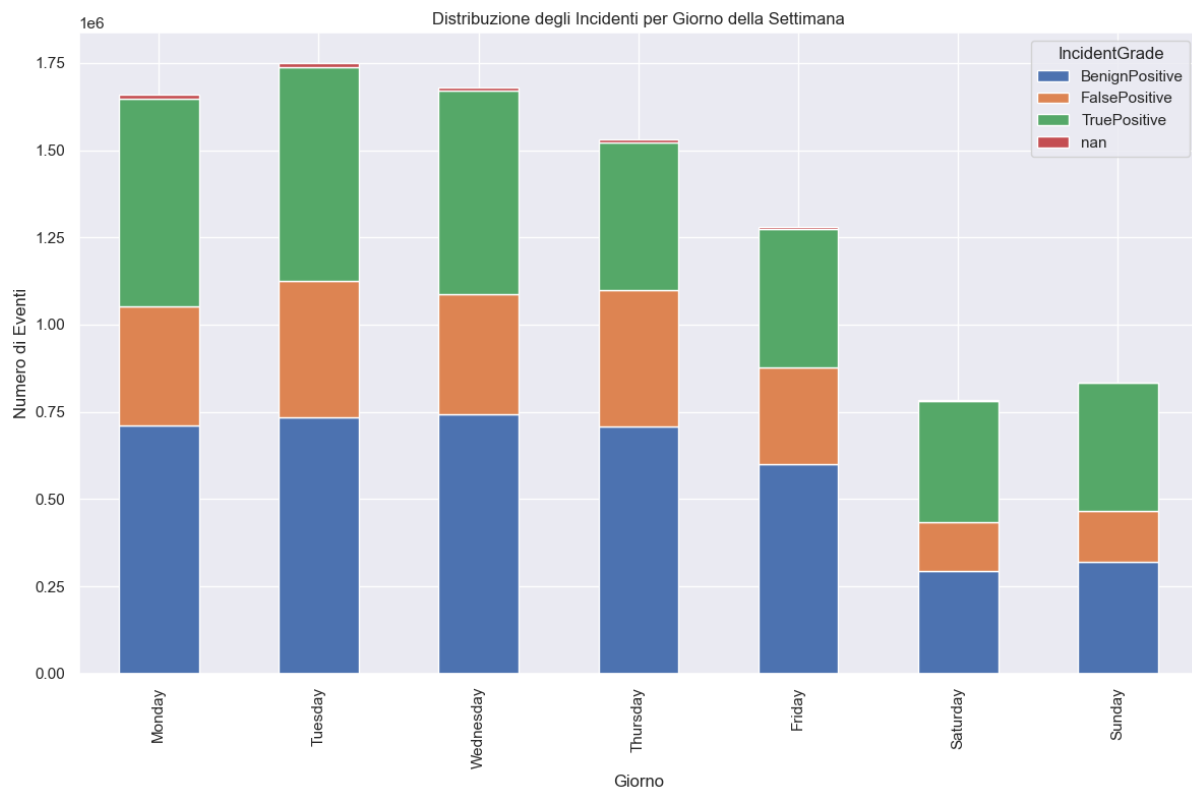


Figura 2.3: Distribuzione degli incidenti per giorno della settimana, suddivisi per IncidentGrade.

La Figura 2.4 illustra la distribuzione oraria degli eventi registrati nell'arco delle 24 ore, distinguendo tra le classi TruePositive e Benign/False Positive.

Dall'analisi del grafico emerge un marcato andamento circadiano sincronizzato con i ritmi delle attività lavorative. Si osserva un punto di minimo volume tra le ore 04:00 e le 07:00 del mattino, seguito da una crescita costante a partire dalle 08:00, in corrispondenza dell'inizio dell'orario d'ufficio. Il picco massimo di attività viene raggiunto nella fascia pomeridiana, tra le 15:00 e le 17:00, per poi decrescere gradualmente durante le ore serali.

È interessante notare come la classe Benign/False Positive presenti una variabilità molto più accentuata rispetto ai TruePositive, i quali mantengono un volume sensibilmente inferiore e una linea di tendenza più stabile, seppur seguendo lo stesso pattern di crescita diurna. Questa discrepanza suggerisce che il traffico legittimo o i falsi allarmi siano fortemente correlati all'intensità delle operazioni umane e di rete tipiche.

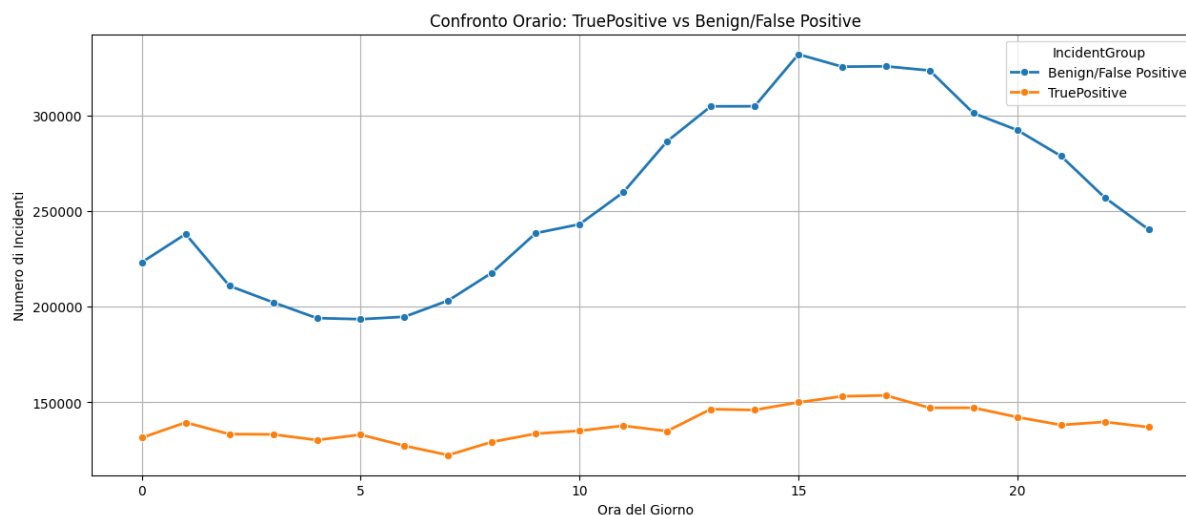


Figura 2.4: Distribuzione degli incidenti per ora del giorno, suddivisi per IncidentGrade.

2.3.3 Distribuzione delle Gravità degli Incidenti

L'analisi della gravità (*Severity*) degli incidenti, visualizzata in Figura 2.5, mostra la ripartizione tra le categorie Bassa, Media, Alta e Informativa. Questa distribuzione aiuta a comprendere il profilo di rischio generale degli incidenti analizzati e può essere correlata con la classificazione di triage per identificare pattern utili.

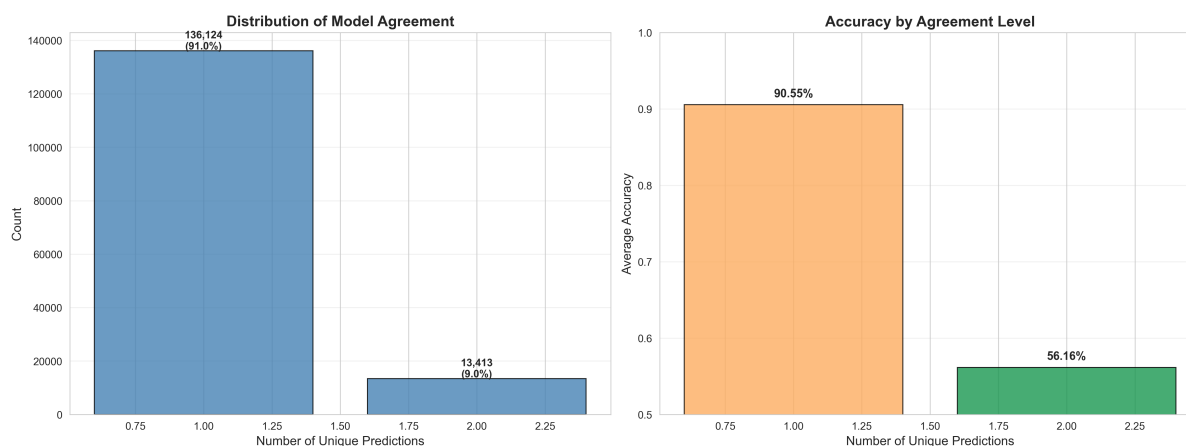


Figura 2.5: Distribuzione della gravità degli incidenti.

2.3.4 Titolo della segnalazione generate

La colonna *AlertTitle* rappresenta l'identificatore categorico del titolo assegnato a ciascun avviso di sicurezza nel dataset. In un contesto operativo reale, questo valore corrisponde alla descrizione sintetica della minaccia rilevata (ad esempio, "Rilevato malware", "Tentativo di accesso brute-force"), ma nel presente dataset è stato rimpiazzato da identificatori numerici per garantire la riservatezza delle logiche di rilevamento proprietarie.

La variabile presenta un'elevatissima cardinalità, indicativa della vastità e della specificità delle euristiche di rilevamento di Microsoft. Di seguito si riportano i dati salienti emersi dall'analisi esplorativa:

- **Cardinalità:** Sono presenti ben **86.149** valori univoci, suggerendo che il sistema monitora una gamma estremamente variegata di comportamenti sospetti.
- **Distribuzione:** Nonostante l'alto numero di titoli unici, la distribuzione è fortemente sbilanciata verso un ristretto gruppo di allerte ricorrenti, tipico dei sistemi di monitoraggio su larga scala dove alcuni segnali di telemetria standard generano la maggior parte del volume di dati.(Figura 2.6)

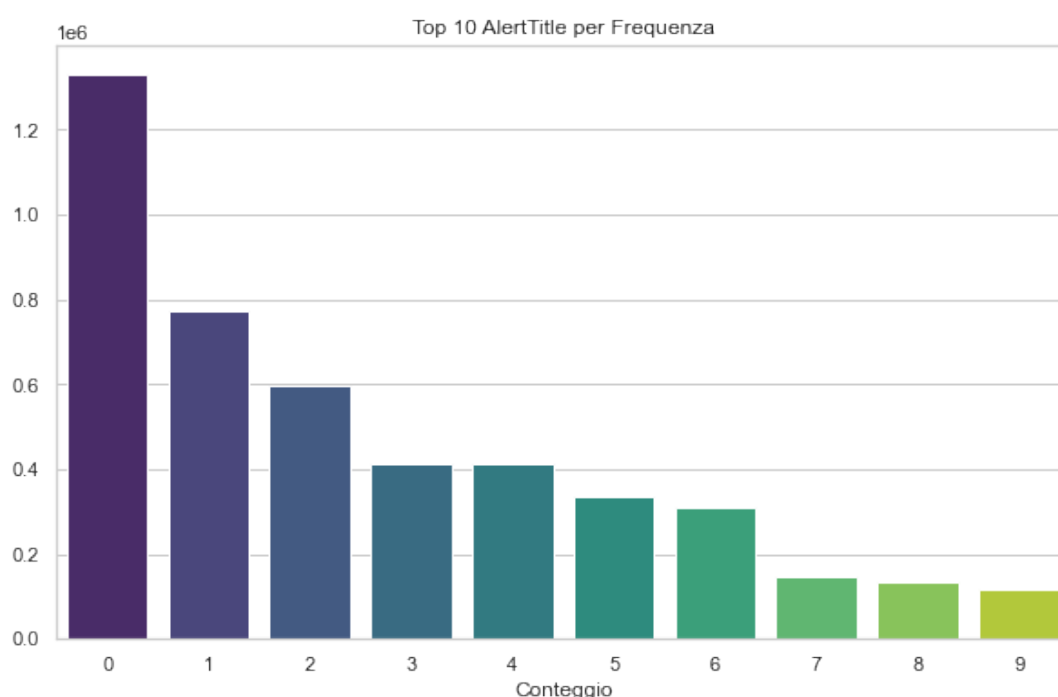


Figura 2.6: Conteggio allertTitle

2.3.5 Frequenza dei Titoli Maggiori

La Tabella 2.2 illustra i dieci titoli di allerta più frequenti nel dataset, che rappresentano i vettori di rilevamento primari in termini di volume.

L'elevato numero di valori unici implica che la variabile `AlertTitle` agisce come una firma granulare. Mentre i titoli con frequenza elevata (come l'ID 0) potrebbero rappresentare eventi comuni o rumore di fondo che richiedono una classificazione accurata, i titoli rari (parte della "lunga coda" degli 86.149 valori) spesso identificano minacce altamente specifiche o anomalie rare che potrebbero avere una probabilità più alta di essere incidenti critici (True Positives).

ID	AlertTitle	Frequenza (Conteggio)
0		133.223
21		77.453
92		59.749
74		41.387
93		41.208
75		33.465
26		30.909
37		14.554
78		13.541
109		11.779

Tabella 2.2: Top 10 AlertTitle per occorrenza nel dataset.

2.3.6 Analisi componenti hardware

Il dataset fornisce informazioni dettagliate sulle entità coinvolte nell'evento. Le colonne `DeviceName`, `FileName`, `IpAddress` e `Url` fungono da indicatori di contesto, permettendo di identificare dove e come l'anomalia si è manifestata. Tutte queste variabili sono state anonimizzate in forma numerica. L'analisi evidenzia una distribuzione "a coda lunga", dove una piccola frazione di entità appare in un numero elevato di record, mentre la stragrande maggioranza compare raramente.

DeviceName: Rappresenta l'endpoint (workstation o server) su cui è stato generato l'avviso. L'elevata cardinalità (oltre 114.000 dispositivi) indica un ambiente enterprise di grandi dimensioni.

FileName: Indica il file o l'eseguibile associato all'allerta. Con 222.085 valori unici, è la feature con la cardinalità più alta, riflettendo la varietà di processi e software presenti sugli endpoint.

IpAddress: Identifica l'indirizzo di rete coinvolto. È un indicatore critico per rilevare tentativi di connessione verso domini di comando e controllo o movimenti laterali nella rete.

Url: Rappresenta le risorse web accedute. Come per gli IP, l'analisi delle URL è fondamentale per identificare attacchi di phishing o download di payload malevoli.

L'assenza di valori nulli in queste colonne suggerisce che ogni allerta nel dataset è corredata da almeno un'entità di riferimento, garantendo una base informativa completa per l'addestramento dei modelli di classificazione.

2.3.7 Analisi geografica

Le variabili CountryCode, State e City forniscono la collocazione geografica dell'asset o dell'evento che ha generato l'allerta. Questi dati sono fondamentali per l'analisi del rischio geo-politico e per l'identificazione di campagne di attacco mirate a specifiche regioni.

2.3.8 Distribuzione Geografica

A differenza delle variabili tecniche (IP o Device), i dati geografici presentano una cardinalità limitata, coerente con le suddivisioni amministrative globali. Le statistiche per queste feature, evidenziando l'assoluta completezza del dato (0% valori nulli). Dall'analisi della distribuzione emerge un forte sbilanciamento verso determinati codici geografici.

2.3.9 Tecniche MITRE ATT&CK più Frequenti

La variabile MitreTechniques rappresenta l'attributo semantico più critico del dataset, in quanto associa le allerte tecniche a specifiche tattiche e tecniche di attacco documentate nel framework MITRE ATT&CK. Questa mappatura trasforma un dato di telemetria grezzo in una conoscenza operativa strutturata, facilitando la comprensione delle intenzioni dell'avversario. L'analisi del dataset rivela una densità informativa peculiare, caratterizzata da un'elevata percentuale di valori mancanti (57.5%).

- **Sparsità del dato:** Su oltre 9,5 milioni di record, circa 4,05 milioni (42.5%) presentano una tecnica mappata. In ambito Cybersecurity, questa "vacuità" non deve essere interpretata come un errore di campionamento, bensì come il riflesso della natura stessa dei sistemi di monitoraggio: non tutte le anomalie rilevate sono riconducibili a pattern di attacco noti o a comportamenti avversari strutturati.
- **Qualità della Rilevazione:** Una copertura del 42.5% indica un sistema di rilevamento maturo, capace di arricchire quasi la metà degli eventi con metadati contestuali di alto livello.

I dati seguono la sintassi ufficiale MITRE, che prevede identificatori univoci (es. T1189) ed eventuali sotto-tecniche (es. T1078.004). Analizzando i campioni forniti, si osservano diverse tipologie di attacco:

Un aspetto fondamentale emerso dall'analisi è la presenza di **valori concatenati** (es. T1559;T1106;T1059.005). Questa caratteristica indica che una singola allerta può attivare molteplici indicatori di compromissione (IoC). La co-occorrenza di più tecniche (come T1098 - Account Manipulation e T1114 - Email Collection) fornisce una traccia della catena di attacco (*Kill*

Codice	Nome Tecnico	Descrizione Operativa
T1189	Drive-by Compromise	Accesso iniziale tramite siti web malevoli che sfruttano vulnerabilità del browser.
T1566	Phishing	Invio di messaggi fraudolenti per indurre l'esecuzione di codice o la sottrazione di credenziali.
T1078.004	Cloud Accounts	Utilizzo di account cloud compromessi per persistenza o escalation di privilegi.
T1114	Email Collection	Tecniche per sottrarre messaggi e-mail dai server o dai client delle vittime.

Tabella 2.3: Esempi di tecniche MITRE identificate nel dataset.

Chain), permettendo di distinguere tra rumore di fondo e attacchi mirati complessi.

2.3.10 Correlazione tra le Variabili Numeriche

Per investigare le relazioni lineari tra le variabili numeriche del dataset, è stata calcolata e visualizzata una matrice di correlazione, mostrata in Figura 2.7. La matrice utilizza una scala di colori dove il rosso indica una forte correlazione positiva (valori vicini a +1), il blu una forte correlazione negativa (valori vicini a -1) e il bianco un'assenza di correlazione lineare (valori vicini a 0).

Dall'analisi emergono alcuni cluster di variabili fortemente correlate. Ad esempio, si nota un'alta correlazione positiva tra `AccountSid`, `AccountUpn` e `AccountObjectId`, il che è atteso poiché sono tutti identificatori univoci dello stesso account utente. Allo stesso modo, variabili come `FileName` e `FolderPath` mostrano una correlazione significativa.

L'identificazione di queste ridondanze è utile in fase di feature selection per evitare problemi di multicollinearità e per ridurre la dimensionalità del modello.

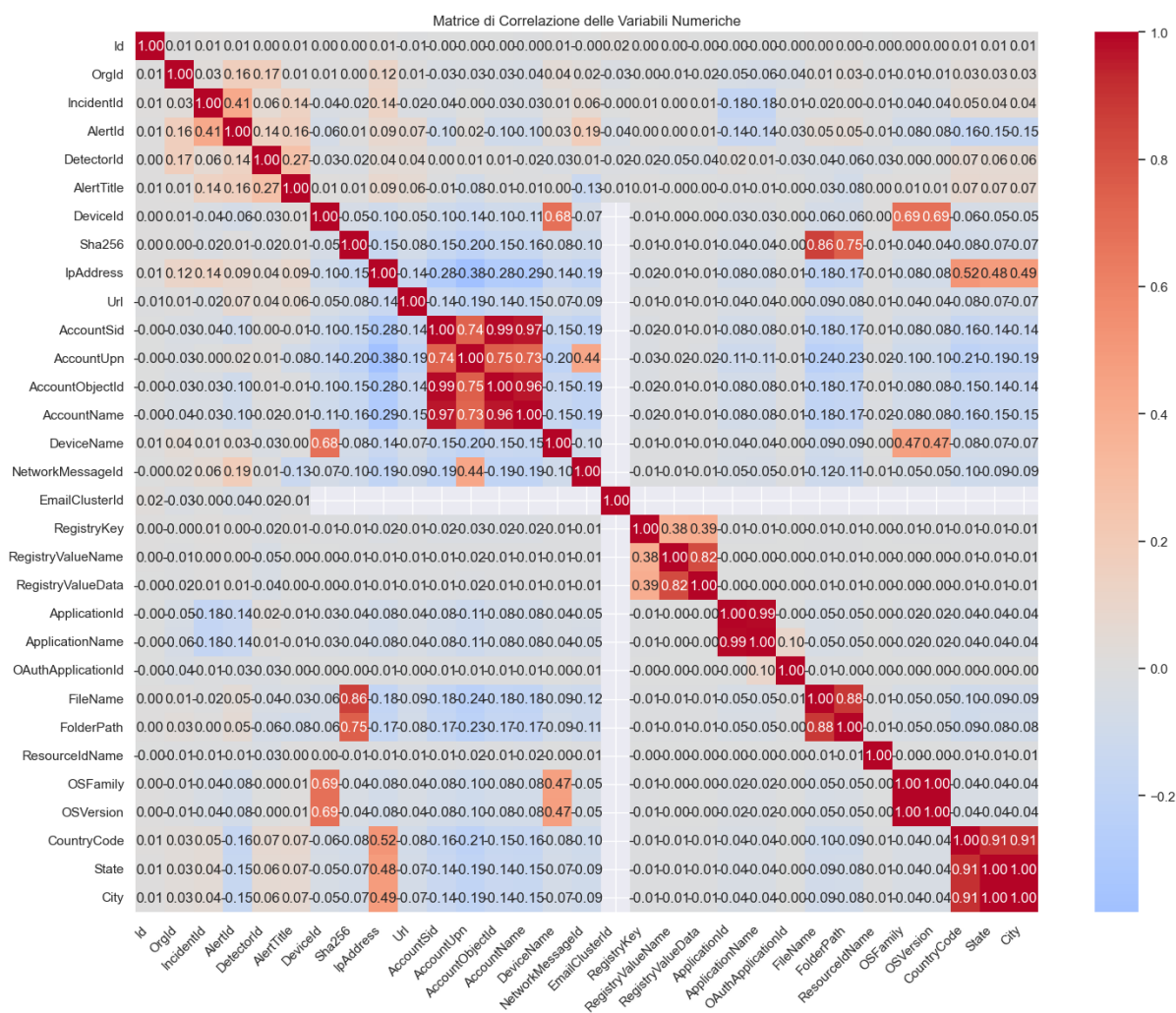


Figura 2.7: Matrice di correlazione tra le variabili numeriche del dataset.

2.4 Strumenti Utilizzati

Per l'analisi dei dati, il preprocessing e lo sviluppo dei modelli predittivi sono stati impiegati i seguenti strumenti, librerie e framework, scelti per la loro robustezza, flessibilità e ampia adozione nella comunità di data science.

- **Python:** Linguaggio di programmazione principale, scelto per il suo vasto ecosistema di librerie per l'analisi dei dati, machine e deep learning.
- **Pandas:** Libreria fondamentale per la manipolazione, la pulizia e l'analisi di dati strutturati. È stata utilizzata per caricare, trasformare e aggregare i dati del dataset GUIDE.
- **Scikit-learn:** La libreria di riferimento per l'implementazione di algoritmi di machine learning. È stata utilizzata per il preprocessing dei dati

(scaling, encoding), l'addestramento dei modelli di classificazione e la loro valutazione tramite metriche e tecniche di convalida incrociata. E' stata usata anche per addestrare una rete MLP semplice sulla CPU.

- **PyTorch:** libreria per python che permette di costruire, addestrare e utilizzare modelli di deep learning, in questo caso usato per un modello di classificazione MLP, per testare eventuali differenze con quello implementato in Scikit-Learn.
- **Matplotlib & Seaborn:** Librerie utilizzate per la creazione di visualizzazioni statiche, come istogrammi, grafici a barre, matrici di correlazione e curve ROC, essenziali per la fase di analisi esplorativa e per la presentazione dei risultati.
- **Jupyter Notebook:** Ambiente di sviluppo interattivo che ha permesso di combinare codice, visualizzazioni e testo descrittivo, facilitando l'esplorazione dei dati e la documentazione del processo di analisi.
- **VSCode e PyCharm:** gli editor/ambienti di sviluppo utilizzati per l'implementazione del codice.

3 Extract - Transform - Load

Il preprocessing dei dati a disposizione è stato cruciale data l'elevata dimensionalità (46 colonne) ed eterogeneità degli stessi. L'obiettivo di questa fase è stato quello di ridurre e trasformare i dati grezzi di GUIDE in un formato pulito, coerente e ottimizzato per addestrare e testare gli algoritmi di classificazione. Le operazioni di pulizia e trasformazione hanno permesso di massimizzare le performance dei modelli, riducendo il rischio di overfitting e garantendo che l'algoritmo apprenda pattern significativi anziché rumore statistico.

3.1 Data Cleaning

Le decisioni prese durante la fase di pulizia sono state guidate dalla necessità di creare un dataset coerente e robusto, eliminando il rumore e gestendo le imperfezioni dei dati grezzi. Eventuali dati

È stata condotta un'analisi dettagliata dei valori mancanti per ogni colonna. In base alla natura della variabile e alla distribuzione dei dati, sono state adottate le seguenti strategie:

- **Dati categorici:** per le colonne categoriche, i valori mancanti sono stati sostituiti con una stringa costante come 'Missing' o 'missing' prima di applicare il frequency o one-hot encoding.
- **Dati numerici:** eventuali valori mancanti sono stati riempiti con il valore -999 tramite `X.fillna(-999)`. Operazione svolta soprattutto nella fase di feature engineering.

3.1.1 Eliminazione delle Colonne

Diverse colonne sono state identificate e rimosse in quanto non fornivano informazioni utili per il task di classificazione o avrebbero potuto introdurre rumore:

- **Identificatori Unici:** queste colonne saranno eliminate durante la fase di feature engineering dato che alcune sono utili per effettuare aggregazioni. Verranno mantenute le colonne degli identificatori
- **Colonne con Alta Percentuale di Valori Mancanti:** feature con una percentuale di valori nulli superiore a una soglia critica (80% in questo caso) sono state scartate, poiché qualsiasi tecnica di imputazione sarebbe stata inaffidabile e avrebbe potuto introdurre artefatti.
- **OrgId** colonna esplicitamente eliminata per evitare possibili overfitting su identificativi categorici.
- Le colonne rimanenti sono **IncidentId, Target, AlertTitle, EvidenceId, DeviceName, FileName, IpAddress, Url, MitreTechniques**.

3.2 Strategia di Splitting

Una delle sfide principali del dataset GUIDE è la sua struttura gerarchica: ogni incidente è composto da molteplici prove (righe). Uno split casuale tradizionale (row-wise) avrebbe distribuito evidenze dello stesso incidente sia nel training che nel validation set, portando il modello a "ricordare" l'incidente piuttosto che ad apprendere pattern generalizzabili. Una situazione che avrebbe portato ad un inquinamento del dataset di training con informazioni del test set, causando una pesante sovrastima delle prestazioni.

Per escludere eventuali *data leakage*, è stata adottata una strategia rigorosa:

- **Split a livello Incident:** l'unità per la suddivisione dei dati è `IncidentId`; tutte le prove appartenenti ad un singolo incidente vengono assegnate in blocco esclusivamente a train o test set.
- **Riallocazione del Test Set:** il dataset di test originale conteneva oltre 500 mila incidenti, una quantità eccessiva per gli scopi prefissati in questo progetto. Per questo motivo è stato mantenuto un test set complessivo di 20 mila campioni, più che sufficienti per validare i modelli implementati.
- **CrossValidation stratificata:** lo split tra training (90%) e validation (10%) è stato eseguito in modo stratificato rispetto alla variabile target, garantendo che la distribuzione delle classi rimanesse coerente.

3.3 Feature Engineering

Prima dell'aggregazione, le singole prove sono state arricchite con nuove feature calcolate in modo da evitare qualsiasi data leak dal validation o test set.

3.3.1 Smoothed Risk Score

La feature `AlertTitle` presenta un alta cardinalità ed una forte capacità predittiva. Piuttosto che usare un semplice *One-Hot encoding*, è stato implementato un *Bayesian Smoothed Risk Score*. Per ogni tipologia di alert, è stato calcolato un "punteggio di rischio" basato sulla probabilità che quell'alert sia associato a un incidente `TruePositive`. La formula utilizzata per lo smoothing è:

$$\text{SmoothedRisk} = \frac{n \cdot \bar{y} + \alpha \cdot \bar{y}_{global}}{n + \alpha}$$

- `n`: numero di occorrenze dell'alert.
- $\bar{y}$: media del target per alert.
- `$\bar{y}_{global}$` : media globale del target nel training set.
- `alpha`: parametro di regolarizzazione (fissato a 5).

Questo approccio evita l'overfitting su alert rari, avvicinando le loro stime verso la media globale. E' importante sottolineare come la mappa di encoding sia stata calcolata esclusivamente sul training set e successivamente applicata al validation e test set.

3.3.2 Frequency Encoding e Geo-Localizzazione

Per le variabili categoriche ad altissima cardinalità come `DeviceName`, `FileName`, `IpAddress` e `Url`, è stato applicato un *Frequency Encoding*, sostituendo ogni categoria con la sua frequenza relativa nel training set. Inoltre, le informazioni geografiche (`CountryCode`, `State`, `City`) sono state concatenate in un'unica feature `GeoLoc`, successivamente codificata in frequenza per catturare pattern regionali di attacco.

3.3.3 Gestione delle Tecniche MITRE ATT&CK

La colonna `MitreTechniques` contiene liste di tecniche (es. "T1078;T1566") separate da punto e virgola. L'analisi ha evidenziato oltre 400 tecniche uniche. Per gestire questa complessità senza esplosione dimensionale:

1. È stato effettuato il parsing delle stringhe per isolare le singole tecniche.
2. Sono state identificate le **Top 20 tecniche** più frequenti nel Training Set.
3. È stato applicato un One-Hot Encoding manuale solo per queste 20 tecniche.
4. È stata creata una feature `Mitre_Count` che rappresenta il numero totale di tecniche associate all'evidenza, preservando l'informazione sulla complessità dell'attacco.

3.4 Aggregazione: Da Evidence a Incident

La fase cruciale dell'ETL è la trasformazione del dataset da "livello evidenza" a "livello incidente". Le righe appartenenti allo stesso `IncidentId` sono state compresse in un unico record utilizzando funzioni di aggregazione specifiche per ogni tipo di feature:

- **Target:** funzione `max`. Se almeno un'evidenza nell'incidente è etichettata come `TruePositive` (1), l'intero incidente è considerato positivo.
- **AlertRisk:** sono stati calcolati `mean` e `max` del rischio degli alert associati, per catturare sia il rischio medio che il picco di pericolosità.
- **Feature Temporal:** media e deviazione standard dell'ora (`Hour`) per capire la distribuzione temporale dell'attacco.
- **Complessità:** conteggio delle evidenze totali (`EvidenceCount`) e degli alert unici (`AlertCount`).
- **Tecniche MITRE:** funzione `max` sulle feature binarie (presenza/assenza della tecnica nell'incidente) e somma del `Mitre_Count`.

3.5 Bilanciamento e Selezione delle Feature

3.5.1 Analisi della Multicollinearità

Prima dell'addestramento, è stata condotta un'analisi della correlazione tra le feature aggregate. Sono state identificate coppie di variabili con correlazione assoluta superiore a 0.9 (es. alcune tecniche MITRE spesso co-occorrenti come `T1078` e `T1078.004`). Sebbene evidenziate, queste feature sono state inizialmente mantenute per permettere ai modelli come XGBoost di selezionare le più informative.

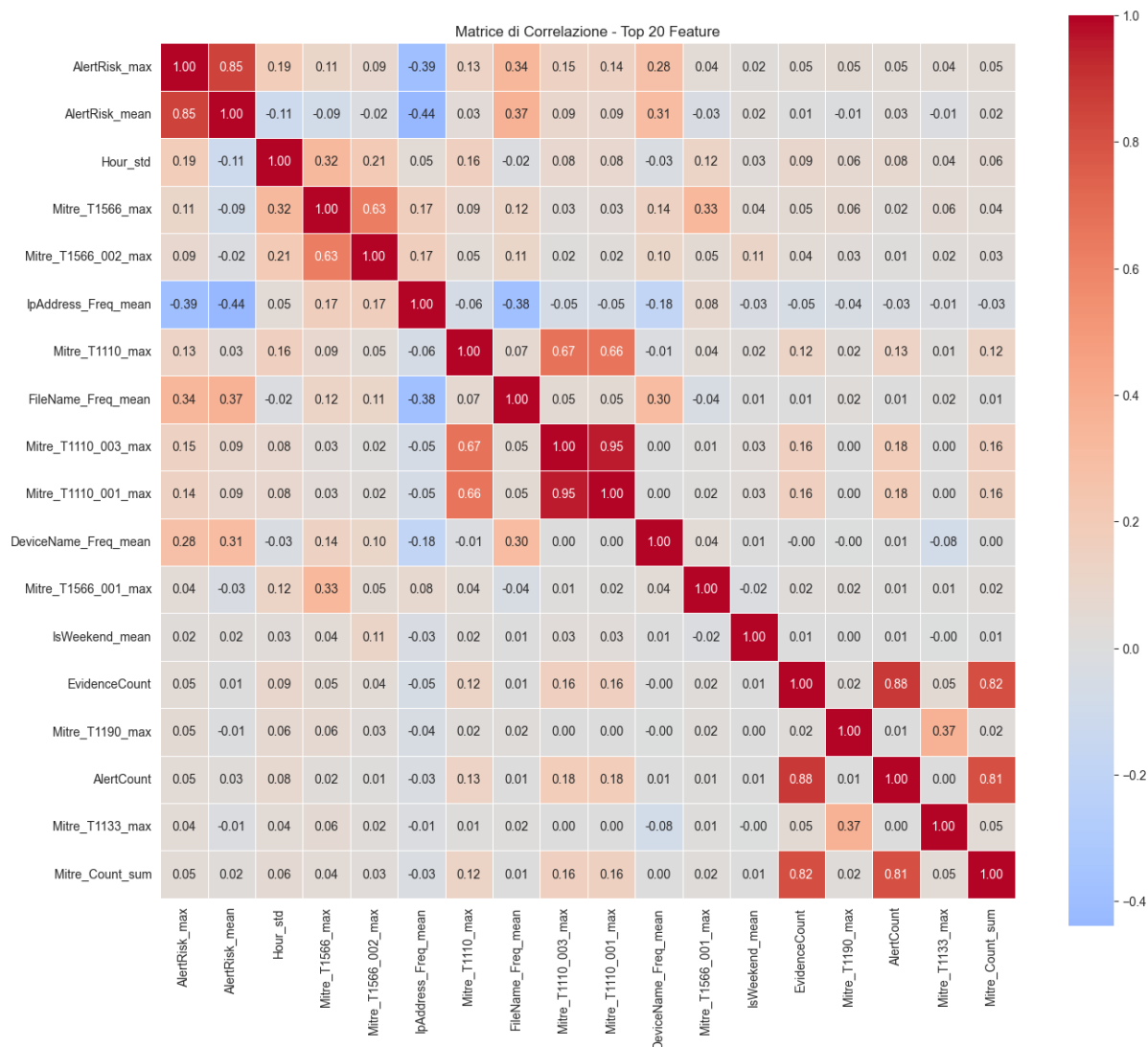


Figura 3.1: Correlazioni delle principali feature con il target.

3.5.2 Gestione dello Sbilanciamento (Undersampling)

Il dataset risultante presentava ancora uno sbilanciamento significativo verso la classe negativa (rapporto circa 3:1); si ricorda che il dataset originale è stato arricchito con alcuni campioni presi dal test set. Dopo un primo approccio con Undersampling e SMOTE, si è optato per un semplice **Random Undersampling** della classe maggioritaria.

Una scelta giustificata dall'ampia disponibilità di dati reali (oltre 150.000 campioni per la classe minoritaria nel training esteso): l'undersampling riduce il rumore e il costo computazionale senza introdurre artefatti sintetici, producendo un training set perfettamente bilanciato (1:1) di circa 300.000 campioni totali.

3.5.3 Scaling delle Feature Numeriche

Infine, tutte le feature numeriche sono state scalate utilizzando lo `StandardScaler`. Questa trasformazione standardizza le feature portandole ad avere una media pari a 0 e una deviazione standard pari a 1. Lo scaling è fondamentale per i modelli basati su reti neurali come il Multi-Layer Perceptron implementato; è stato comunque applicato per uniformità al dataset per tutti i modelli poiché le prestazioni di Alberi Decisionali, XGBoost e Random Forest non vengono influenzati da questa modifica.

3.6 Risultati della Fase di Preprocessing

Il risultato finale è un set di dati robusto che pone solide fondamenta per l'addestramento e la valutazione di modelli predittivi efficaci, tenendo sempre presente la sfida posta dallo sbilanciamento delle classi, che verrà affrontata nel capitolo successivo.

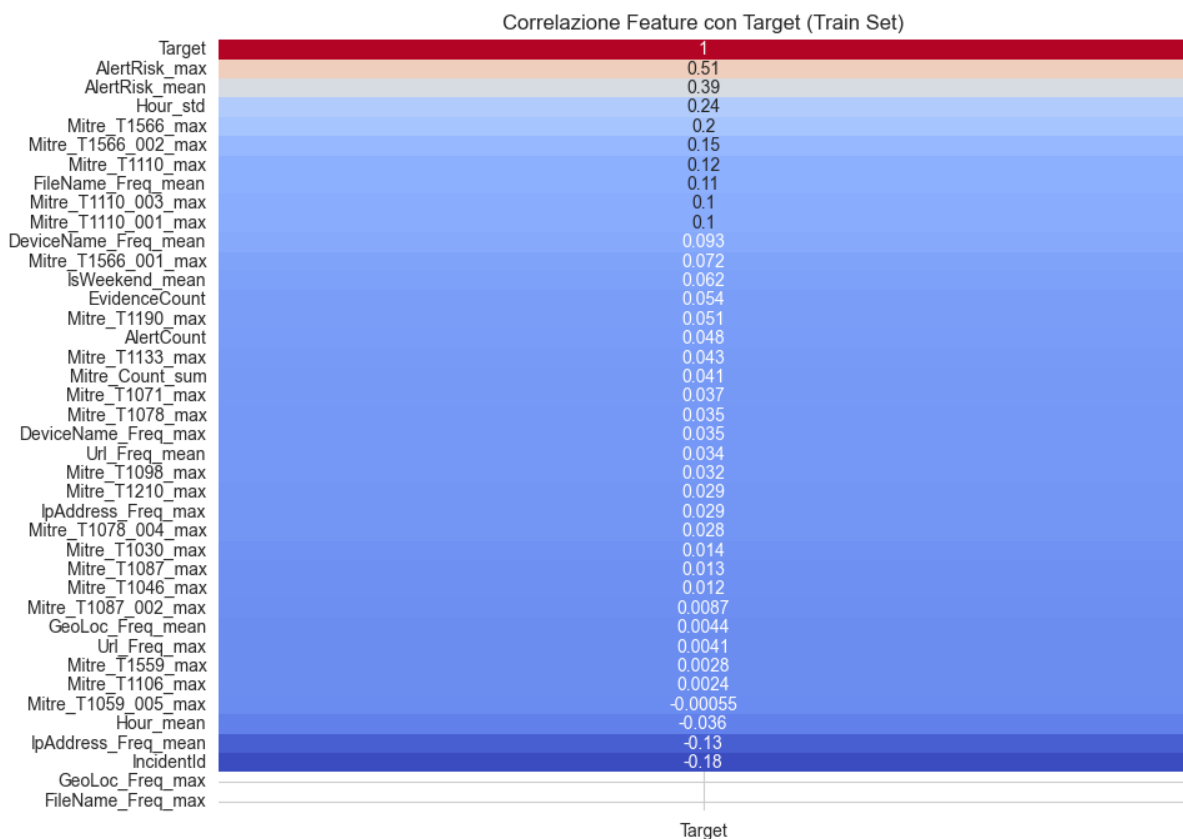


Figura 3.2: Correlazioni delle principali feature con il target.

4 Classificazione

Una volta ottenuto un dataset pulito, strutturato e non eccessivamente numeroso, la fase successiva e centrale del progetto si è concentrata sullo sviluppo, l'ottimizzazione e la valutazione rigorosa di diversi modelli di Intelligenza Artificiale.

L'obiettivo di questo capitolo è descrivere il processo di selezione degli algoritmi, confrontare sistematicamente le loro performance e analizzare in profondità i risultati per identificare l'approccio più efficace per la classificazione automatica del triage degli incidenti di sicurezza.

4.1 Metodologia di Valutazione

Per garantire una valutazione oggettiva e robusta, è stata definita una metodologia standard applicata a tutti i modelli.

- **Suddivisione del Dataset:** il dataset è stato suddiviso in un set di addestramento, validazione e test, come indicato nel capitolo precedente.
- **Validazione:** per l'ottimizzazione degli iperparametri, è stata utilizzata una strategia di Hold-Out per ottenere una stima stabile delle performance e ridurre il rischio di overfitting, suddividendo i dati di addestramento in 90% per training effettivo e 10% validazione. È stata preferita questa tecnica per l'elevata quantità di dati a disposizione per l'addestramento dei modelli.
- **Metriche di Valutazione:** nota l'originale natura sbilanciata del dataset, è stata scelta come metrica principale la **F1-Score**, mentre per visualizzare le prestazioni a livello grafico la curva **ROC AUC**.

4.2 Modelli di Classificazione

Data la natura del task e la numerosità del dataset, sono stati scartati i modelli della famiglia "Nearest - Neighbor": sarebbe infatti risultato difficile

e computazionalmente oneroso riuscire ad utilizzarli con oltre 300 mila record.

Poiché l'obiettivo dei modelli è quello di distinguere un rischio reale da un falso allarme, il numero di categorie è stato ridotto da 3 a 2, accorpando "benign_positive" e "false_positive" in un'unica label, come anticipato nella sezione sulla *feature engineering*.

Dunque le scelte sono ricadute su quattro tipologie di modelli: *Decision Tree*, *Random Forest*, *XGBoost* e *Multi-Layer Perceptron*.

4.2.1 Tuning degli Iperparametri

La metodologia adottata per ottenere migliori prestazioni è stata l'applicazione della `RandomizedSearchCV` con la 5-Fold Stratified Cross Validation.

Anche se inizialmente era stata pensata una `GridSearchCV`, l'opzione adottata ha permesso di lavorare in modo più efficiente su uno spazio più ampio; inoltre, consente di fissare un "budget" computazionale, in termini di numero di iterazioni.

4.2.2 Decision Tree

L'Albero Decisionale è un algoritmo di classificazione basato sullo split iterativo dei dati in funzione delle feature ed è stato implementato attraverso la libreria `sklearn`. Fin dai primi test ha dimostrato buone prestazioni sul validation set.

```
1 dt_param_dist = {
2     'max_depth': [5, 10, 15, 20, None],
3     'min_samples_split': [2, 5, 10, 20],
4     'min_samples_leaf': [1, 2, 4, 8]
5 }
6
7 dt_search = RandomizedSearchCV(
8     DecisionTreeClassifier(random_state=42),
9     dt_param_dist, n_iter=20, cv=cv, scoring='f1',
10     ↪ n_jobs=-1, random_state=42
11 )
12 dt_search.fit(X_train, y_train)
```

Nella porzione di codice riportata nel paragrafo precedente sono evidenti i parametri del modello, in particolare:

- **max_depth**: utile per limitare la profondità dell'albero, riduce il rischio di overfitting.
- **min_samples_split**: minimo sample per split.
- **min_samples_leaf**: minimo sample per foglia.

Al termine del training sono stati ottenuti i valori delle metriche riportati nella tabella di figura 4.1.

```
DecisionTree - Validation F1: 0.8299
DecisionTree - Validation AUC: 0.9123
Params: {'min_samples_split': 20, 'min_samples_leaf': 1, 'max_depth': 15}
```

Figura 4.1: Metriche di classificazione dell'albero decisionale.

4.2.3 Random Forest

La Random Forest è un algoritmo di *ensemble learning* che costruisce una moltitudine di alberi decisionali in fase di training e restituisce la classe che è la moda delle classi dei singoli alberi. È stato implementato attraverso la libreria `sklearn`.

```

1  rf_param_dist = {
2      'n_estimators': [50, 100, 150],
3      'max_depth': [10, 15, 20],
4      'min_samples_split': [2, 5, 10],
5      'min_samples_leaf': [1, 2, 4]
6  }
7
8  rf_search = RandomizedSearchCV(
9      RandomForestClassifier(n_jobs=1, random_state=42),
10     rf_param_dist, n_iter=15, cv=cv, scoring='f1',
11     n_jobs=-1, # Parallelizza solo CV, non ogni RF
12     random_state=42,
13     verbose=1 # Mostra progresso
14 )
15
16 rf_search.fit(X_train, y_train)
```

Nella porzione di codice riportata nel paragrafo precedente sono evidenti i parametri assegnati al modello, in particolare:

- **n_estimators**: il numero di alberi nella foresta.
- **max_depth**: la profondità massima di ogni albero.

- **min_samples_split**: il numero minimo di campioni richiesto per split-tare un nodo interno.
- **min_samples_leaf**: il numero minimo di campioni richiesto per essere in un nodo foglia.

Al termine del training sull'ultimo dataset sono stati ottenuti i valori delle metriche riportati nella tabella di figura 4.2.

```
RandomForest - Validation F1: 0.8391
RandomForest - Validation AUC: 0.9291
Params: {'n_estimators': 150, 'min_samples_split': 5, 'min_samples_leaf': 2, 'max_depth': 20}
```

Figura 4.2: Metriche di classificazione della Random Forest.

4.2.4 XGBoost

L'XGBoost (eXtreme Gradient Boosting) è un algoritmo di machine learning basato sull'idea del boosting, in cui vengono creati in sequenza dei modelli deboli (solitamente alberi decisionali) in modo che ogni nuovo modello corregga gli errori del precedente. È noto per le sue elevate prestazioni e la sua velocità di esecuzione, ed è stato implementato tramite la libreria `xgboost`.

```

1  xgb_param_dist = {
2      'max_depth': [4, 6, 8, 10],
3      'learning_rate': [0.01, 0.05, 0.1, 0.2],
4      'n_estimators': [100, 200, 300],
5      'min_child_weight': [1, 3, 5],
6      'subsample': [0.8, 0.9, 1.0],
7      'colsample_bytree': [0.8, 0.9, 1.0]
8  }
9
10 xgb_search = RandomizedSearchCV(
11     XGBClassifier(eval_metric='logloss', n_jobs=-1,
12     ↪ random_state=42),
13     xgb_param_dist, n_iter=30, cv=cv, scoring='f1',
14     ↪ n_jobs=-1, random_state=42
15 )
16
17 xgb_search.fit(X_train, y_train)
```

Nella porzione di codice riportata nel paragrafo precedente sono evidenti i parametri assegnati al modello, in particolare:

- **n_estimators**: il numero di alberi (passaggi di boosting) da eseguire.
- **max_depth**: la profondità massima di un albero per controllare la complessità.
- **learning_rate**: il tasso di apprendimento, che riduce il contributo di ciascun albero per prevenire l'overfitting.
- **subsample**: la frazione di campioni da utilizzare per l'addestramento di ciascun albero.
- **colsample_bytree**: la frazione di feature da utilizzare per la costruzione di ciascun albero.

Al termine del training sull'ultimo dataset sono stati ottenuti i valori delle metriche riportati nella tabella di figura 4.3.

```
XGBoost - Validation F1: 0.8468
XGBoost - Validation AUC: 0.9338
Params: {'subsample': 0.9, 'n_estimators': 300, 'min_child_weight': 1, 'max_depth': 10, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
```

Figura 4.3: Metriche di classificazione di XGBoost.

Nella sezione relativa alla comparazione dei modelli è riportato il grafico delle feature utilizzate dall'XGBoost.

4.2.5 Multi-layer Perceptron

Il Multi-layer Perceptron (MLP) è una classe di rete neurale artificiale *feed-forward*, implementata in questo caso attraverso il classificatore MLP disponibile come modulo della libreria `sklearn`.

```

1  mlp_model = MLPClassifier(
2      hidden_layer_sizes=(128, 64),
3      activation='relu',
4      solver='adam',
5      alpha=0.001,
6      batch_size=256,
7      learning_rate='adaptive',
8      learning_rate_init=0.001,
9      max_iter=200,
10     random_state=42,
11     early_stopping=True,
12     validation_fraction=0.1,
13     n_iter_no_change=10
14 )
15
```

L'architettura definita nel codice precedente è composta da una sequenza di strati (layers), ognuno con uno scopo specifico:

- **Architettura:** viene utilizzata la ReLU come funzione di attivazione, la cui non linearità permette alla rete di apprendere pattern complessi.
- **Training:** per migliorare l'apprendimento è stato scelto un learning rate adattivo con ottimizzatore ADAM.
- **Dispositivo:** la rete MLP costruita con questa libreria viene addestrata ed eseguita su CPU.

Al termine del training sull'ultimo dataset sono stati ottenuti i valori delle metriche riportati nella tabella di figura 4.4.

```
Addestramento MLP (Sklearn)...  
MLP - Validation F1: 0.8107  
MLP - Validation AUC: 0.9023  
MLP - Epochs completed: 26
```

Figura 4.4: Metriche di classificazione del Multi-layer Perceptron.

Implementazione Pytorch

Come ulteriore esperimento sulla rete MLP, è stata nuovamente implementata utilizzando PyTorch: questo framework permette di beneficiare di hardware ad-hoc per l'addestramento e l'inferenza dei modelli di Deep Learning. Viene riportato di seguito il codice del grafo del classificatore MLP; per regolarizzare il modello è stato introdotto il dropout sui diversi livelli della rete.

```
1  
2     class IncidentMLP(nn.Module):  
3         def __init__(self, input_dim):  
4             super(IncidentMLP, self).__init__()  
5             self.layers = nn.Sequential(  
6                 nn.Linear(input_dim, 128),  
7                 nn.ReLU(),  
8                 nn.BatchNorm1d(128),  
9                 nn.Dropout(0.3),  
10  
11                 nn.Linear(128, 64),  
12                 nn.ReLU(),  
13                 nn.BatchNorm1d(64),  
14                 nn.Dropout(0.2),
```

```

15
16         nn.Linear(64, 32),
17         nn.ReLU(),
18         nn.BatchNorm1d(32),
19
20         nn.Linear(32, 1)
21     )
22
23     def forward(self, x):
24         return self.layers(x)
25

```

Le prestazioni ottenute nel test set ottenuto con hold-out (in Figura 4.5) ed in quello condiviso da tutti i modelli (in Figura 4.6).

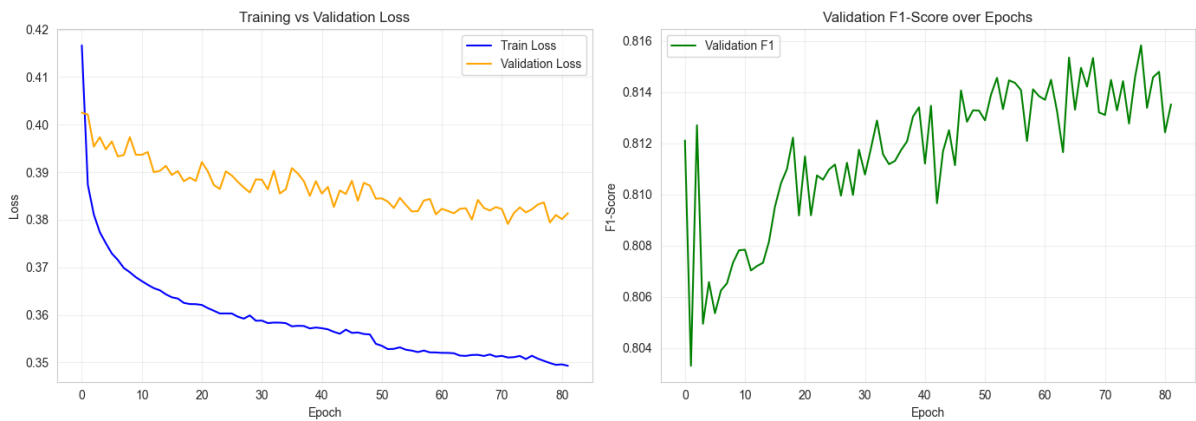


Figura 4.5: Curve di apprendimento del modello.

Si può osservare come le prestazioni non siano migliori di quelle ottenute dal classificatore MLP di `sklearn`, anzi addirittura peggiorano, non beneficiando dell'addestramento su GPU.

Osservando le curve di apprendimento, i dati utilizzati per l'addestramento sono sufficienti, è l'architettura stessa che non ha raggiunto le prestazioni desiderate: in questo caso prevale una rete MLP più semplice e ottimizzata dalla libreria `scikit-learn`.

```

=====
RISULTATI FINALI PYTORCH
=====
Accuracy: 0.7654
F1-Score: 0.7651
ROC AUC: 0.8752

```

Figura 4.6: Prestazioni sul test set

- **CUDA**: per i sistemi dotati di schede grafiche Nvidia con cuDNN 12.4 e compatibili. In questo caso è stata utilizzata una RTX 3060Ti con 8GB di VRAM.
- **MPS**: specifico per sistemi Apple con hardware AMD Radeon e M-Series, testato su M4 Pro.
- **CPU**: ultimo stadio del fallback, utilizza la CPU come dispositivo per il training (e l'inferenza) solo nel caso in cui non siano disponibili i due precedenti. È importante sottolineare che si avrà un'elaborazione molto più lenta.

Pertanto, si suggerisce di utilizzare CUDA o MPS per replicare i risultati ottenuti in questa trattazione.

4.3 Valutazione Comparativa dei Modelli

Sono stati addestrati e valutati quattro diversi algoritmi: Decision Tree, Random Forest, XGBoost e MLP. La Figura 4.7 offre una panoramica completa delle loro performance su tutte le metriche chiave.

	Accuracy	Precision	Recall	F1-Score	AUC
Model					
DecisionTree	0.786800	0.807658	0.781929	0.794585	0.885370
RandomForest	0.791900	0.825201	0.768086	0.795620	0.899208
XGBoost	0.801850	0.827432	0.788755	0.807631	0.906128
MLP	0.767850	0.810149	0.731108	0.768602	0.872016

Figura 4.7: Confronto delle performance dei quattro modelli su cinque metriche chiave: Accuracy, Precision, Recall, F1-Score e ROC AUC. Si osserva come i modelli basati su alberi (Random Forest, XGBoost) tendano a sovraperformare il modello MLP.

Come si evince dal confronto, i modelli basati su ensemble di alberi (Random Forest e XGBoost) mostrano una superiorità generale. Per approfondire, la Figura 4.8 si concentra sulla metrica ROC AUC, che misura la capacità di un modello di distinguere tra le classi.

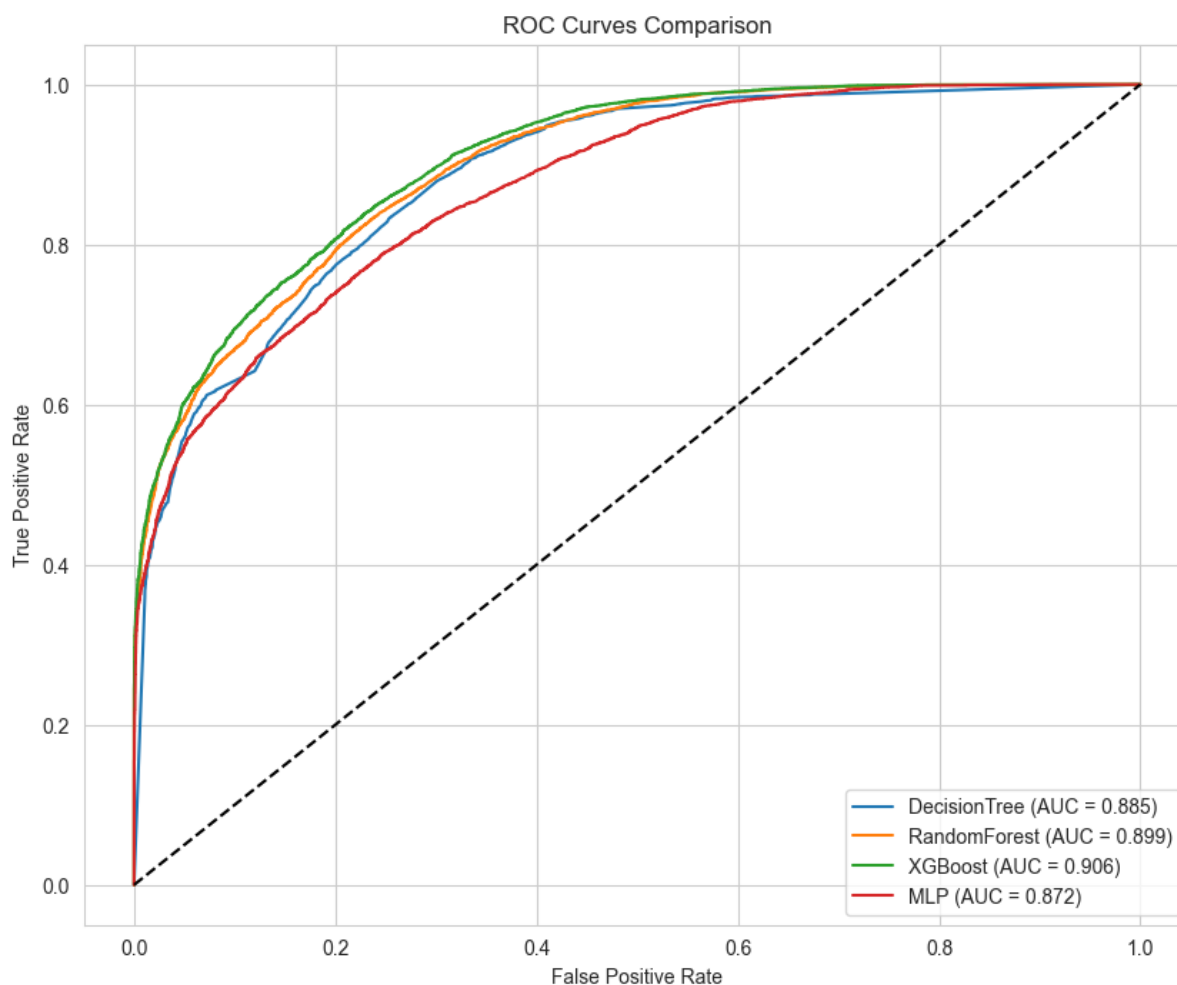


Figura 4.8: Confronto dei punteggi ROC AUC per i modelli binari. XGBoost emerge come il modello con la più alta capacità discriminante (AUC = 0.9304), seguito da vicino da Random Forest e Decision Tree.

4.4 Analisi Dettagliata delle Performance

Per comprendere non solo quanto bene i modelli performano, ma anche come commettono errori, sono state analizzate le matrici di confusione. La Figura 4.9 confronta il comportamento di tre dei modelli principali.

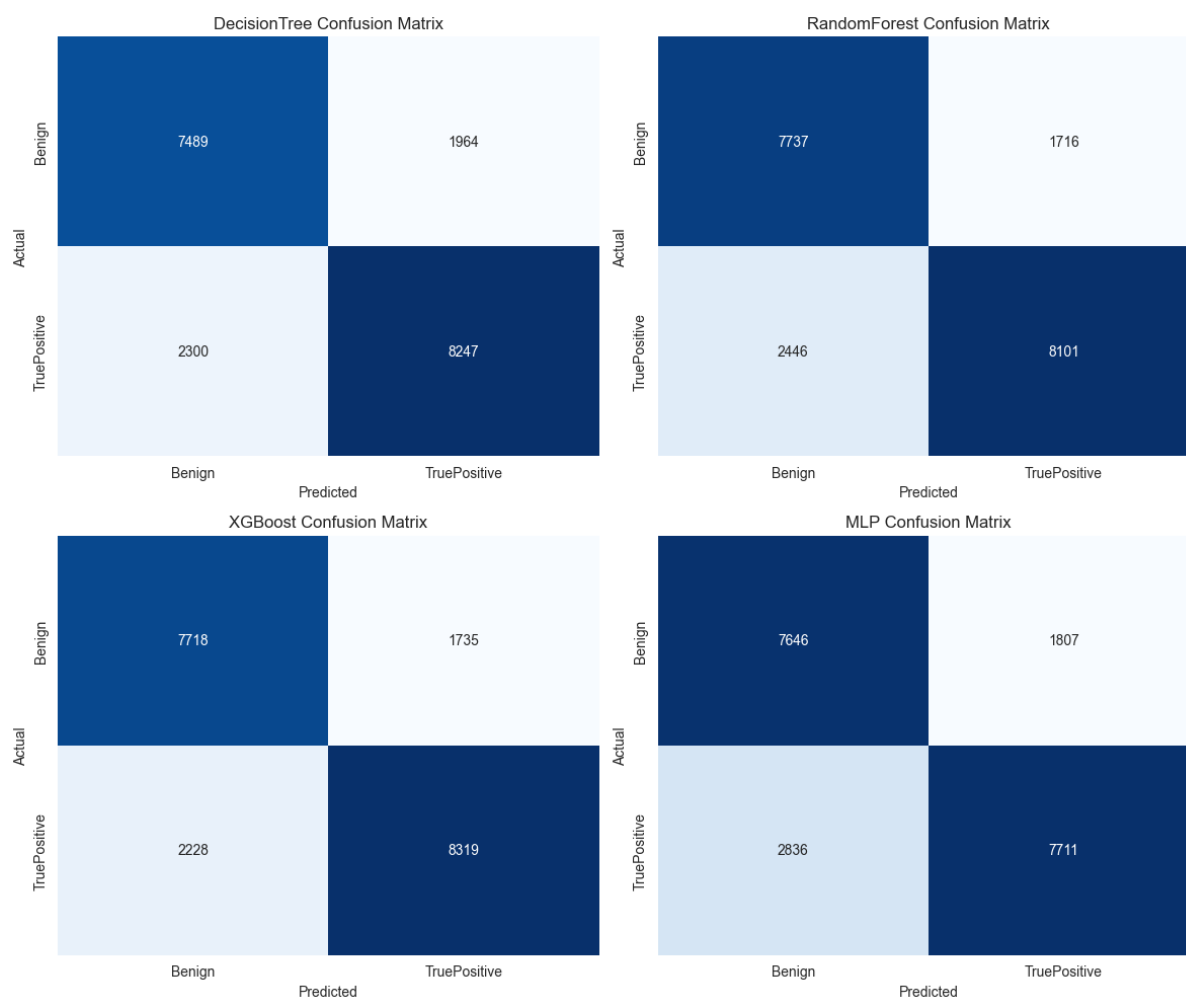


Figura 4.9: Matrici di confusione per i modelli Random Forest, Decision Tree, XGBoost e MLP.

4.5 Analisi di Interpretabilità: Feature Importance

Capire quali feature influenzano maggiormente le decisioni di un modello è cruciale per guadagnare fiducia nei suoi risultati e per ottenere insight sul fenomeno studiato. La Figura 4.10 mostra le 10 feature più importanti per il modello XGBoost, scelto anche per la sua capacità di "selezionare" le feature più informative.

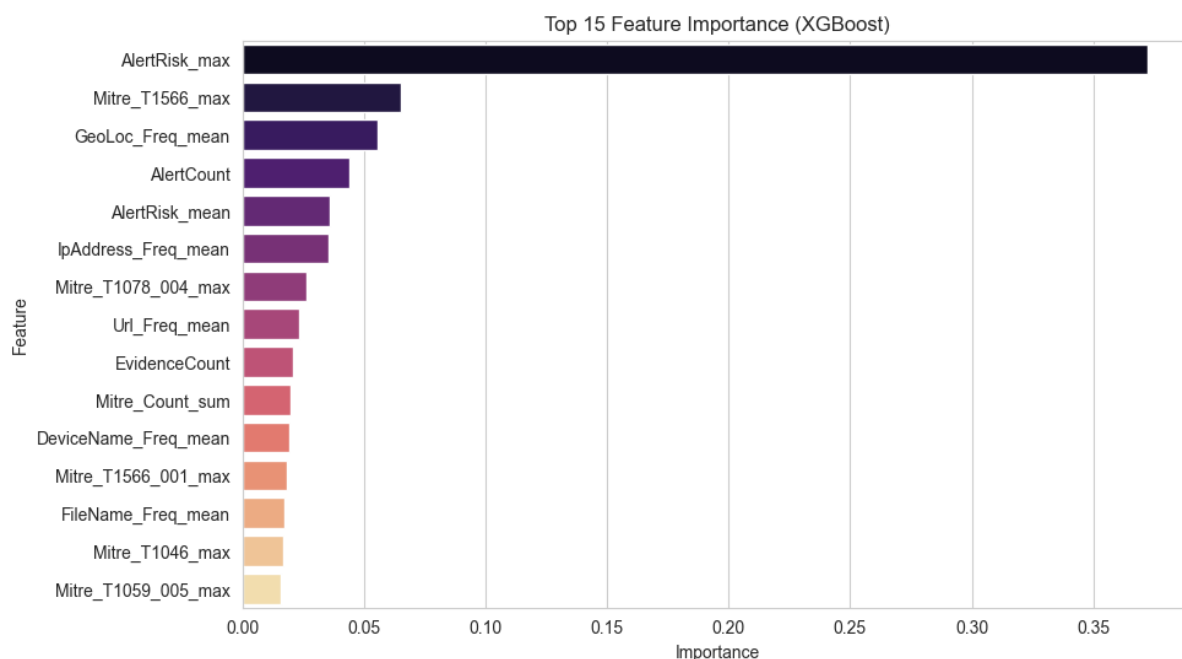


Figura 4.10: 10 feature più importanti secondo il modello XGBoost.

L'importanza delle feature estratta dal modello XGBoost permette di interpretare i driver decisionali dell'algoritmo. La distribuzione dell'importanza è caratterizzata da una variabile dominante, seguita da indicatori semantici e comportamentali.

4.5.1 Predominanza di AlertRisk_max

La feature `AlertRisk_max`, derivata dallo *Smoothed Target Encoding* della colonna `AlertTitle`, risulta essere il predittore più significativo con un valore di importanza prossimo a 0.37.

- **Significato:** Questo risultato conferma che la tipologia intrinseca dell'allerta è l'indicatore primario di pericolosità. Il fatto che la versione *max* (il rischio massimo associato alle allerte aggregate in un incidente) sia superiore alla *mean* suggerisce che basta un singolo evento ad alto rischio all'interno di un gruppo per determinare la natura malevola dell'intero incidente.
- **Efficacia dello Smoothing:** L'elevata importanza valida la scelta metodologica dello smoothing: il modello è in grado di estrarre segnali puliti anche da titoli ad alta cardinalità.

4.5.2 Rilevanza delle Tecniche MITRE ATT&CK

Diverse variabili legate al framework MITRE compaiono tra le prime 15 feature, sottolineando come la natura "tattica" dell'evento sia fondamentale:

- **Mitre_T1566_max (Phishing):** Si posiziona come la seconda feature più importante. Ciò indica che le allerte riconducibili a campagne di phishing hanno una probabilità statistica molto elevata (e molto chiara per il modello) di essere incidenti confermati.
- **Mitre_T1078_004_max (Cloud Accounts):** La presenza di tecniche legate alla compromissione di account cloud evidenzia un vettore di attacco critico nel dataset Microsoft GUIDE.

4.5.3 Indicatori Comportamentali e di Contesto

Il modello utilizza variabili di frequenza e conteggio per identificare anomalie statistiche:

- **GeoLoc_Freq_mean:** L'importanza della frequenza geografica suggerisce che la rarità (o la ripetitività) di un evento in una determinata area è un forte discriminante.
- **AlertCount:** Il volume di allerte generate simultaneamente è un segnale di allarme; attacchi complessi tendono a innescare più sensori contemporaneamente. rischio.

4.5.4 Conclusioni sul Modello

Il modello dimostra una forte capacità di astrazione: non si limita a memorizzare ID grezzi, ma sfrutta le **trasformazioni di rischio** (*AlertRisk*) e le **mappature semantiche** (*MITRE*) per generalizzare il rilevamento. La netta distanza tra *AlertRisk_max* e le altre feature suggerisce che il sistema di sicurezza è ben calibrato: i titoli delle allerte assegnati dai sistemi Microsoft sono già di per sé estremamente descrittivi del verdetto finale.

4.6 Valutazione Finale del Modello

Sulla base dell'analisi precedente, il modello finale scelto è XGBoost, così come è stato definito nel rispettivo paragrafo. Questo modello si è dimostrato superiore, sebbene con un margine sottile, rispetto agli altri candidati, in particolare rispetto alla Random Forest che ha mostrato performance quasi altrettanto valide. La scelta si fonda su diverse considerazioni chiave emerse durante la valutazione comparativa:

- **Performance superiori:** come evidenziato in figura ??, XGBoost ha ottenuto il punteggio più alto in due delle metriche più significative per il nostro task. Ha mostrato il miglior **F1-Score** (0.8076), confermando un equilibrio ottimale tra precisione e recall, fondamentale per minimizzare sia i falsi positivi che i falsi negativi. Inoltre, ha raggiunto il mi-

glior **ROC AUC score** (0.9060), indicando una capacità discriminante superiore nel distinguere correttamente tra le due classi.

- **Efficienza e scalabilità:** XGBoost è noto per la sua implementazione ottimizzata che garantisce alta velocità di addestramento e inferenza. Sebbene non sia stato un fattore critico con il dataset attuale, la sua efficienza lo rende una scelta più robusta e scalabile per future applicazioni dove il volume dei dati potrebbe aumentare significativamente.
- **Gestione dell'overfitting:** grazie a tecniche di regolarizzazione integrate (come L1 e L2), XGBoost offre un controllo più granulare contro l'overfitting rispetto ad altri modelli basati su alberi. Questo contribuisce a una maggiore generalizzazione delle performance su dati non visti.
- **Interpretabilità:** nonostante la sua complessità, XGBoost fornisce meccanismi robusti per l'analisi dell'importanza delle feature (come mostrato in figura ??), permettendo di comprendere quali variabili influenzano maggiormente le predizioni del modello. Questa trasparenza è cruciale per generare fiducia nel modello e per ottenere insight pratici dal processo di classificazione.
- **Inefficacia degli ensemble complessi:** i tentativi di creare un modello ensemble, combinando XGBoost con Random Forest o MLP, non hanno prodotto miglioramenti significativi da giustificare l'aumento di complessità. Anzi, in alcuni casi le performance sono addirittura peggiorate. Questo ha ulteriormente rafforzato la decisione di optare per il singolo modello più performante.

Dunque, XGBoost si è rivelato il miglior compromesso tra accuratezza predittiva, robustezza e interpretabilità per il task di classificazione in esame. La sua capacità di ottenere le migliori performance sulle metriche chiave, unita alla sua efficienza computazionale, lo rende la scelta ottimale per l'implementazione finale del sistema.

5 Conclusioni

Questo progetto ha affrontato la sfida complessa e di fondamentale importanza della classificazione automatica degli incidenti di cybersecurity. In un contesto operativo dove i Security Operations Centers (SOC) sono sommersi da un volume insostenibile di avvisi, l'automazione intelligente del processo di triage non è più un'opzione, ma una necessità strategica. Attraverso un'analisi approfondita del dataset su larga scala GUIDE e l'applicazione di una rigorosa pipeline metodologica di machine learning, sono stati sviluppati e validati modelli predittivi in grado di assistere efficacemente gli analisti, migliorando l'efficienza e l'efficacia della risposta agli incidenti.

5.1 Riepilogo e Interpretazione dei Risultati

Il modello finale sviluppato, XGBoost, ha dimostrato una notevole capacità nel predire le etichette di triage, raggiungendo un **F1-Score di 0.8076**. Questo risultato indica un solido equilibrio tra la capacità di identificare correttamente le minacce reali (TP) e di filtrare gli attacchi non pericolosi e finti allarmi (Non-TP). Le tecniche di ensemble non si sono rivelate particolarmente efficaci, rispetto al singolo XGBoost che aveva già ottenuto eccellenti risultati.

5.1.1 La Sfida dei "True Positive"

L'analisi delle performance ha evidenziato in modo inequivocabile che la classificazione degli incidenti "True Positive" (TP) rappresenta la sfida maggiore. Questo risultato non è sorprendente. La loro corretta identificazione rimane un'area chiave per futuri miglioramenti e richiederà probabilmente l'ingegnerizzazione di feature più sofisticate o l'uso di modelli in grado di catturare un contesto ancora più ricco. In questo senso, utilizzare un modello Deep Learning in grado di catturare relazioni complesse potrebbe essere la soluzione, già il MLP utilizzato in questa trattazione ha raggiunto risultati accettabili, seppur inferiori all'XGBoost: si potrebbe ipotiz-

zare la necessità di un'architettura più profonda per catturare pattern più complessi.

5.1.2 Dinamicità del Dominio e Concept Drift

Una considerazione critica per l'applicazione pratica di questo lavoro è la natura dinamica della cybersecurity. Le tattiche, tecniche e procedure (TTP) degli avversari evolvono costantemente per eludere i sistemi di difesa. Ciò implica che qualsiasi modello predittivo, addestrato su dati storici, è inevitabilmente soggetto a "concept drift": il suo potere predittivo degraderà nel tempo man mano che emergono nuove minacce non viste in precedenza. Per questo motivo, un sistema di classificazione di questo tipo non può essere statico. La sua implementazione in un ambiente di produzione richiederebbe un framework di MLOps (Machine Learning Operations) che includa un monitoraggio continuo delle performance e un meccanismo di riaddestramento periodico con dati recenti e annotati, garantendo così la sua efficacia nel lungo periodo.

5.2 Contributi Principali del Progetto

Questo progetto ha permesso di esplorare un complesso dataset di cybersecurity del mondo reale e di applicare un flusso di lavoro completo di Data Science, fornendo diversi contributi significativi:

- La progettazione e l'implementazione di una **pipeline di preprocessing robusta e riproducibile**, in grado di gestire dati di sicurezza eterogenei e rumorosi, inclusa l'elaborazione di feature complesse e multi-label come le tecniche del framework MITRE ATT&CK.
- Lo **sviluppo e il confronto sistematico** di diversi modelli di classificazione, che ha permesso di validare empiricamente come gli approcci di ensemble basati su alberi decisionali (Random Forest, LightGBM) siano superiori ai modelli lineari per questo tipo di task.
- Il raggiungimento di un **solido benchmark di performance** sul dataset pubblico GUIDE. Questo risultato può servire come un valido punto di riferimento per futuri lavori di ricerca e per la valutazione di nuove tecniche sulla stessa base dati.
- L'analisi descrittiva e interpretativa delle feature, ha fornito **insight qualitativi** su quali caratteristiche (es. specifici `DetectorId`, combinazioni di tecniche MITRE, o la gravità degli avvisi) siano più predittive di una minaccia reale, informazioni preziose per gli analisti di sicurezza.

5.3 Sviluppi Futuri e Prospettive di Ricerca

Sebbene i risultati ottenuti siano incoraggianti, esistono ampi e promettenti margini di miglioramento. Le direzioni future per la ricerca potrebbero includere:

- **Deep Learning per Dati Sequenziali:** Gli incidenti di sicurezza sono spesso una sequenza di eventi. L'esplorazione di architetture di deep learning come le Reti Neurali Ricorrenti (RNN) o i Transformer potrebbe permettere di modellare le dipendenze temporali tra gli avvisi all'interno di un incidente, catturando pattern che i modelli statici non possono vedere.
- **Graph Neural Networks (GNNs):** La struttura gerarchica del dataset (prove → avvisi → incidenti) si presta naturalmente a una rappresentazione a grafo. L'utilizzo di Graph Neural Networks potrebbe rivelarsi estremamente potente per modellare esplicitamente le complesse relazioni tra le entità, apprendendo non solo dalle loro feature, ma anche dalla topologia delle loro connessioni.
- **Integrazione di Threat Intelligence Esterna:** Arricchire il dataset con fonti di dati esterne, come feed di threat intelligence su indirizzi IP malevoli, hash di file noti o domini di phishing, potrebbe fornire un contesto cruciale e migliorare significativamente la capacità del modello di identificare minacce emergenti.
- **Sistemi Human-in-the-Loop:** Invece di un'automazione completa, un'area di ricerca promettente è lo sviluppo di sistemi "human-in-the-loop", dove il modello fornisce raccomandazioni e probabilità all'analista, e allo stesso tempo impara dal feedback e dalle correzioni fornite da quest'ultimo. Questo approccio sinergico consentirebbe di ottenere dati estremamente precisi per un nuovo addestramento.

In conclusione, questo lavoro conferma il potenziale del machine learning come strumento per potenziare le capacità dei SOC. I modelli sviluppati rappresentano un passo concreto verso la realizzazione di sistemi di "risposta guidata" intelligenti, in grado di automatizzare il triage di routine e di consentire agli analisti umani di concentrare la loro insostituibile expertise sulle minacce più sofisticate e critiche.

La repository GitHub del progetto è disponibile al seguente link: <https://github.com/DataScience-Golddiggers/Dora-the-Data-Explorer.git>

